

Optimal Logarithmic-Time Selection: A Formal Analysis of the Median of Two Independently Sorted Arrays

Anonymous Researcher

Abstract—This paper presents a rigorous theoretical analysis and algorithmic formulation for finding the median of two independently sorted arrays in $O(\log(\min(m, n)))$ time. We formulate the selection problem as an optimal partition search, proving the exact boundary invariants required for median resolution without linear-time merging. Through detailed mathematical proofs, we establish the correctness of the constrained binary search, define the strict boundary conditions, and analyze the asymptotic bounds of both spatial and temporal complexities. The findings formally confirm that the generalized median selection problem over partitioned, totally ordered sets operates strictly within a logarithmic bounded search space.

$\log(\min(m, n))$

time. We formulate the selection problem as an optimal partition search, proving the exact boundary invariants required for median resolution without linear-time merging. Through detailed mathematical proofs, we establish the correctness of the constrained binary search, define the strict boundary conditions, and analyze the asymptotic bounds of both spatial and temporal complexities. The findings formally confirm that the generalized median selection problem over partitioned, totally ordered sets operates strictly within a logarithmic bounded search space.



1 Introduction

2 Introduction

The identification of the exact median element across multiple sorted datasets is a foundational query in algorithm design and distributed systems. When merging parallel data streams, sensor logs, or executing distributed database queries, computing the median natively requires a $O(m + n)$ linear merge operation. However, leveraging the independent total ordering of each array enables the transformation of the search space into an optimized binary search problem. This paper provides a complete mathematical formalization of the $O(\log(\min(m, n)))$ algorithm.

3 Mathematical Problem Formulation

Let X and Y be two arrays of elements belonging to a totally ordered domain $\mathbb{D}$. Array X has length m and array Y has length n . Both X and Y are monotonically non-decreasing such that $\forall k, x_k \leq x_{k+1}$ and $y_k \leq y_{k+1}$.

Let Z be the sorted multiset union of X and Y , possessing length $N = m + n$. The median of Z , denoted as $\mathcal{M}$, is defined conditionally based on the parity of N :

$$\mathcal{M} = \begin{cases} Z[\frac{N-1}{2}] & \text{if } N \text{ is odd} \\ \frac{1}{2} (Z[\frac{N}{2} - 1] + Z[\frac{N}{2}]) & \text{if } N \text{ is even} \end{cases} \quad (1)$$

Our objective is to compute $\mathcal{M}$ without explicitly constructing Z , satisfying a strict upper temporal bound of $O(\log(\min(m, n)))$ and spatial bound of $O(1)$.

4 The Partition Invariant Theorem

To avoid a linear merge, we define a structural partition across both arrays. We aim to identify an index $i \in [0, m]$ to partition X and an index $j \in [0, n]$ to partition Y , such that the combined left segments equal the combined right segments in size.

Let the left partition L and right partition R be defined as:

$$L = X[0 \dots i - 1] \cup Y[0 \dots j - 1] \quad (2)$$

$$R = X[i \dots m - 1] \cup Y[j \dots n - 1] \quad (3)$$

To ensure that the partition splits the conceptual array Z exactly at the median, the partition indices must satisfy the Length Invariant:

$$i + j = \lfloor \frac{m + n + 1}{2} \rfloor \quad (4)$$

Furthermore, to guarantee that all elements in L precede all elements in R in the combined sorted order, the Value Invariant must hold:

$$\max(L) \leq \min(R) \quad (5)$$

Given that X and Y are internally sorted, $X[i - 1] \leq X[i]$ and $Y[j - 1] \leq Y[j]$ trivially hold. Therefore, the Value Invariant reduces to two cross-boundary conditions:

$$X[i - 1] \leq Y[j] \quad \text{and} \quad Y[j - 1] \leq X[i] \quad (6)$$

When both the Length Invariant and the Value Invariant are satisfied, the optimal partition is found.

5 Binary Search Convergence Algorithm

To achieve logarithmic time complexity, we perform a binary search strictly on the smaller array. Without loss of generality, assume $m \leq n$. If $m > n$, we trivially swap references to X and Y prior to execution to prevent index out-of-bounds errors on Y when determining j .

The search space for i is initialized as $[i_{\min}, i_{\max}] = [0, m]$. During each iteration, we select the midpoint:

$$i = \lfloor \frac{i_{\min} + i_{\max}}{2} \rfloor \quad (7)$$

By the Length Invariant, j is uniquely determined as:

$$j = \lfloor \frac{m + n + 1}{2} \rfloor - i \quad (8)$$

We then evaluate the Value Invariant conditions:

- 1) If $X[i - 1] > Y[j]$, then partition i is too far right. We must decrease i by updating $i_{\max} = i - 1$.
- 2) If $Y[j - 1] > X[i]$, then partition i is too far left. We must increase i by updating $i_{\min} = i + 1$.
- 3) If $X[i - 1] \leq Y[j]$ and $Y[j - 1] \leq X[i]$, the optimal partition is identified.

6 Edge Case Resolution and Median Computation

When the optimal indices (i, j) are found, extreme edge cases arise when a partition is empty ($i = 0, i = m, j = 0$, or $j = n$). We resolve this by extending the domain $\mathbb{D}$ with $-\infty, +\infty$:

$$X[-1] = -\infty, \quad X[m] = +\infty \quad (9)$$

$$Y[-1] = -\infty, \quad Y[n] = +\infty \quad (10)$$

Upon invariant satisfaction, the maximum element of the left partition is:

$$L_{\max} = \max(X[i - 1], Y[j - 1]) \quad (11)$$

If N is odd, the median is strictly $L_{\max}$.

If N is even, the median requires the minimum element of the right partition:

$$R_{\min} = \min(X[i], Y[j]) \quad (12)$$

The median is then computed as the arithmetic mean:

$$\mathcal{M} = \frac{L_{\max} + R_{\min}}{2} \quad (13)$$

7 Asymptotic Complexity Proof

Temporal Complexity: The algorithm binary searches over the index space of the smaller array. The search domain size is initially $m + 1$. Since the domain halves each iteration, the maximum number of iterations evaluates to $\lceil \log_2(m + 1) \rceil$. Thus, the upper bound time complexity is strictly $O(\log(\min(m, n)))$.

Spatial Complexity: The structural variables $i_{\min}, i_{\max}, i$, and j are stored as scalar integers. The algorithm requires no auxiliary recursive stack allocation or dynamic memory instantiation. Thus, the spatial complexity is $O(1)$, representing the theoretical lower bound.

8 Conclusion

This paper provided a rigorous theoretical proof for the optimal $O(\log(\min(m, n)))$ algorithm that resolves the median of two independently sorted datasets. By transitioning the problem from element comparison to partition index evaluation, the algorithm satisfies all strict asymptotic constraints and bypasses traditional linear-time array aggregation.

References

- [1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to algorithms (3rd ed.). MIT Press.
- [2] Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.